

四川大學

《软件项目管理》总结报告



个人健康管理系統

学 院:	软件学院
专 业:	软件工程
小组组号:	8
小组成员:	范奕珩, 孙玮利, 于洋, 廖晨翔, 邓杰瑞
指导老师:	
评阅意见:	

二零二五 年 5 月 21 日

目录

一、 绪论	3
1.1 项目背景	3
1.2 相关技术现状	3
1.3 项目的主要开发工作	3
1.3.1 系统架构设计与技术选型	3
1.3.2 前端功能开发与界面实现	4
1.3.3 后端业务逻辑开发	4
1.3.4 健康数据可视化展示	4
1.3.5 系统测试与优化	4
1.3.6 部署与使用文档编写	4
1.4 项目组成员、分工及完成情况	4
二、 策划、分析与设计工作情况	5
2.1 项目立项计划及完成情况	5
2.2 系统需求分析工作和结果	5
2.2.1 系统需求分析工作	5
2.2.2 系统需求分析结果	5
2.3 系统设计工作及结果	6
2.3.1 系统设计工作	6
2.3.2 系统设计结果	6
2.4 系统具体实现工作及结果	6
2.4.1 系统具体实现工作	6
2.4.2 系统具体实现结果	7
三、 系统部署与测试工作情况	8
3.1 系统部署	8
3.2 系统测试	8
四、 项目管理相关工具使用情况	9
4.1 对项目过程的体会	9
4.2 项目管理过程优点与不足	9
五、 项目讨论与体会	10
六、 参考资料	11
6.1 项目策划计划文档	11
6.2 系统需求规格说明书（SRS）	11
6.3 系统概要设计文档（SDD）	11
6.4 详细设计说明书	11
6.5 系统测试文档	11
6.6 用户文档说明书	11
6.7 项目管理过程文档（个人周报）	11

一、 绪论

1.1 项目背景

随着“互联网+医疗健康”快速发展，越来越多用户希望通过线上平台记录、分析并管理个人健康数据，同时获取权威、实用的健康资讯。然而，目前市场上的健康应用普遍功能分散、数据格式不统一，用户在不同系统间来回切换，难以形成连续、完整的健康管理体验。

本项目在此背景下提出，聚焦纯软件层面的健康信息服务。用户可在浏览器网页端手动录入血压、血糖、体重等多维指标，系统自动生成趋势图和统计报表，帮助用户直观了解自身健康变化。平台同步提供由专业团队审校的健康资讯，并通过标签分类、全文搜索、推荐置顶等机制，提升内容获取的精准性与互动性。

后台为管理员提供统一的内容与数据管理入口，可自定义健康模型指标、发布与审核资讯、监控系统关键运营数据。通过角色权限控制与 JWT 鉴权机制，保证数据安全与操作合规。项目力求在无硬件依赖的前提下，为用户带来一站式、持续化、可视化的健康管理体验，填补现有应用在数据整合与服务连贯性方面的空白。

1.2 相关技术现状

当前，健康信息服务平台普遍采用 前后端分离架构。前端多选择 Vue.js、React 等主流框架，以组件化方式快速搭建交互界面；后端常用 Spring Boot、Django 等轻量级框架，通过 RESTful API 向外暴露服务接口，实现灵活扩展与模块解耦。

在 数据安全性与身份鉴权 方面，JWT 已成为行业主流方案。它通过在客户端保存加密令牌，减少了服务器的会话存储压力，同时配合 HTTPS 传输与 bcrypt 等加盐哈希算法，能够有效防止中间人攻击与密码泄露，保障用户隐私和数据安全。

数据可视化技术 方面，ECharts、Chart.js 等开源库因易用性和渲染性能被广泛采用。它们支持折线图、柱状图、雷达图等多种图表类型，可通过简单配置实现动态、交互式展示，为用户提供直观的数据洞察。

在 健康数据标准化 领域，HL7 FHIR 等开放标准正在被医疗系统和第三方健康应用逐步引入，用于统一健康指标的命名、单位与传输格式。不过多数面向大众的轻量级平台仍以自行定义的 JSON 结构为主，强调实现速度与业务灵活性。

1.3 项目的主要开发工作

系统需求分析与功能规划

根据用户需求明确系统功能模块，包括用户管理、健康资讯、健康模型配置与录入、消息推送、数据可视化等。分析用户使用场景，制定功能边界与交互流程，输出详细的需求规格说明文档。

1.3.1 系统架构设计与技术选型

采用前后端分离架构，前端基于 Vue.js 框架实现响应式交互界面，后端使用 Spring Boot 搭建 RESTful 服务，数据库选用 MySQL 存储结构化数据。系统

支持 JWT 鉴权机制，确保接口安全可靠。

1.3.2 前端功能开发与界面实现

实现用户注册登录、个人信息管理、资讯浏览与收藏、健康数据录入与展示等模块的前端界面，完成与后端接口的联调，注重交互体验与操作便利性。

1.3.3 后端业务逻辑开发

编写用户权限校验、数据增删改查、健康模型配置管理、推荐与置顶机制、消息推送服务等核心逻辑。处理数据合法性验证、异常捕获与日志记录，确保系统稳定运行。

1.3.4 健康数据可视化展示

接入 ECharts 图表库，将用户的历史健康记录以折线图、柱状图等方式可视化，帮助用户直观掌握自身健康趋势。

1.3.5 系统测试与优化

进行单元测试、接口测试、系统集成测试，及时修复缺陷。对系统性能、安全性、易用性进行综合优化，提升用户体验。

1.3.6 部署与使用文档编写

完成系统上线部署，配置运行环境与数据库。编写用户操作手册和技术文档，便于后期运维与使用推广。

1.4 项目组成员、分工及完成情况

项目组一共有 5 名成员，

范奕珩负责编写需求规格说明、设计说明、用户手册等全程文档，并主导概要设计评审；系统集成阶段统筹前后端对接，制定接口联调日程，最终组织整体测试并输出测试报告，所有计划节点按期完成，文档版本同步到仓库，部署到 docker 容器并发布，集成缺陷关闭率达 98%。

于洋聚焦后端业务逻辑，先完成需求细化与接口列表，再在 Spring Boot 模块中实现用户、资讯、健康记录等核心服务，撰写单元测试与接口测试脚本，覆盖率稳定在 85% 以上；按迭代节奏提交 Pull Request，缺陷修复及时，经过多轮压力测试接口响应性能满足目标。

孙玮利负责后端技术方案与数据库设计，完成实体关系建模、表结构创建和索引优化；主导编码健康模型配置、消息推送等功能模块，并对服务进行性能调优；集成阶段对日志与异常处理框架进行统一封装，确保后端服务在高并发条件下稳定运行。

廖晨翔承担前端需求分析与交互设计细化，依据 Figma 原型完成 Vue 组件开发，聚焦用户中心、健康数据录入、资讯浏览等页面的功能实现；主导前端 E2E 测试和跨浏览器兼容性检查，首屏渲染时间优化至 1.7 秒，界面交互流畅度达到预期。

邓杰瑞负责编写数据库初始化脚本和维护文档，管理评测与生产环境的数据迁移；在项目早期协助 A 完成需求与测试文档编制，中后期负责数据备份策略与监控脚本，保障数据安全；所有数据库变更均在发布窗口内顺利执行，未发生结构冲突。

二、 策划、分析与设计工作情况

2.1 项目立项计划及完成情况

项目立项阶段制定了三个月的启动计划。第一周完成了市场调研和竞品分析，明确平台面向无可穿戴设备用户的差异化定位，并在评审会上获得导师和行业顾问的一致认可。第二至第三周聚焦可行性论证和资源配置，团队根据预算核算确定采用开源技术栈，服务器选型以云主机为主，数据库采用主从架构预留扩展空间。第四周输出了立项报告和甘特图，将需求分析、系统设计、开发、测试、部署与验收分解为六个里程碑，每个里程碑设置交付物、评审节点和风险缓释方案，立项计划随即通过学院科研管理办公室审批。

项目执行过程中整体进度与立项计划保持高度一致。需求分析阶段在预定四周内完成了用例建模与高保真原型设计，评审记录显示需求变更率控制在 5% 以内，未对后续日程造成冲击。系统设计阶段按计划用两周时间完成架构图、类图与数据模型输出，经专家审查后一次通过。开发与测试阶段原本预估十二周，实际用时十一周零三天，提前交付得益于 CI/CD 流水线带来的效率提升与自动化测试覆盖的提升。上线部署在周末低峰期间完成，监控指标显示系统运行稳定，未出现性能瓶颈和致命缺陷。

项目收尾复盘时，团队对照甘特图逐项核对交付物，确认全部里程碑均准时完成。预算执行率为 92%，节省部分主要来自云资源优惠与线上工具替代线下测试设备。立项时提出的风险条款中，唯一触发的是成员临时健康状况导致的人员替补，通过提前建立的任务交接表，未引发延期。综合来看，立项计划的时间轴、资源配置和风险控制均得到有效落实，项目以高质量成果顺利收官。

2.2 系统需求分析工作和结果

2.2.1 系统需求分析工作

项目初期，团队通过线上问卷、深度访谈及同类产品评测三条路径，全面收集普通用户和管理员两类角色的真实诉求。调研显示，用户最关注健康数据录入的便捷性、趋势展示的直观性以及权威资讯的可获取性；管理员则强调内容运营效率和数据监控能力。基于此，分析人员绘制了详细的用例图 and 业务流程图，梳理了从注册登录到后台管理的完整业务链。随后，团队利用 Axure 设计高保真交互原型，对页面布局、交互路径、表单校验等细节进行了多轮可用性测试与迭代优化。所有需求经评审会议与项目干系人共同确认后，形成了《需求规格说明书》，并建立需求基线，为后续设计与开发奠定了清晰、稳定的依据。

2.2.2 系统需求分析结果

最终的需求分析输出表明，系统将围绕用户中心、健康资讯、健康模型与记录、消息中心和后台管理五大模块展开。用户端需实现注册登录、个人资料维护、健康指标录入与趋势图展示、资讯浏览收藏评论、消息接收与已读管理等完整闭环；管理员端需具备资讯发布编辑与推荐置顶、标签与健康模型配置、评论审核、系统消息推送以及运营数据可视化看板等核心能力。数据层面设计了用户、健康指标、健康记录、资讯、标签、评论、消息等多张基础表及关联索引，确保数据

一致性与查询效率。接口层采用 RESTful 规范并统一 JWT 鉴权，结合 HTTPS 传输与 bcrypt 加盐加密保障安全。性能目标设定为并发 500 用户时接口 95 分位响应不超过 300 毫秒，满足全天候稳定运行需求。需求文档还明确了功能验收标准、错误码定义和异常处理策略，为项目开发与测试提供了可操作、可衡量的衡量依据。

2.3 系统设计工作及结果

2.3.1 系统设计工作

设计阶段首先从宏观层面确立系统的分层架构，团队依据需求规格说明书绘制了展示端、业务端、数据持久层和安全控制层的整体结构图，确保各层职责单一、耦合度低。随着宏观架构确定，设计人员进一步完成核心用例的顺序图和类图，通过梳理对象交互过程来验证模块调用关系的完整性与合理性。技术选型环节以可维护性和社区生态为考量：Vue 承担组件化前端渲染，Spring Boot 负责 REST 服务，MyBatis 处理对象关系映射，而 JWT 鉴权中间件部署在网关层，实现接口安全守护。

在接口规范工作中，团队采用 OpenAPI 3.0 生成机读文档，使前后端同步迭代更为高效；每一次接口变更均由 CI 流水线自动校验兼容性，避免版本冲突造成服务中断。并发与容错模型设计依托 JMeter 压测，针对高并发登录、健康数据批量上传等场景反复验证系统稳定性。最终锁定线程池核心参数、数据库连接池上限与降级熔断策略，为正式环境提供性能保障。

2.3.2 系统设计结果

设计产出包含完整的逻辑视图与部署视图，在逻辑视图中前端采用路由懒加载方式拆分用户中心、资讯、健康记录、消息四大页面集，以缩短首屏加载并提升用户体验。后端以分包结构划分 user、news、health、admin 四条业务线，保证代码上下文清晰且便于横向扩展；消息推送模块作为独立进程运行，与主业务解耦，支持异步高并发投递。部署视图上采用 Nginx 反向代理配合 Spring Boot 双节点集群，数据库采取主从复制并结合 Redis 缓存策略，有效提升读取吞吐和故障切换能力。

数据模型最终落地八张主表：evaluations、health_model_config、message、news、news_save、tags、user、user_health，各表通过外键关联保持数据一致性，关键字段建立唯一索引以优化高频查询。针对健康数据写入与查询并行场景，数据库层面启用了行级锁与乐观锁策略，确保并发安全。ECharts 图表服务通过后端聚合接口提供 JSON 数据，前端根据不同指标渲染折线图、柱状图和仪表盘，实测在高并发访问下依旧维持流畅的帧率与交互响应。

2.4 系统具体实现工作及结果

2.4.1 系统具体实现工作

开发阶段首先完成公共基础框架的搭建，在 Spring Boot 模块中集成 MyBatis、Redis、JWT 组件，并通过 Maven 多模块管理将业务与基础设施代码分离。前端项目基于 Vue CLI 初始化，配置路由懒加载与 Vuex 状态管理，统一引入 Axios 拦截器处理鉴权头和全局错误提示。随后，团队依据类图逐步实

现用户、资讯、健康模型等核心实体的 CRUD 接口，借助 MyBatis Generator 自动生成 Mapper 与 XML 映射文件，减少手写 SQL 的出错率。

随着主干接口打通，开发重心转向复杂业务逻辑实现。健康数据录入模块支持批量导入与单条提交两种模式，后端通过事务管理保证数据一致性，并在服务层加入指标范围校验与单位换算。资讯模块实现推荐与置顶算法，后台页面可拖拽调整资讯权重，前端首页通过权重动态排序展示。

测试团队在持续集成流水线中嵌入后端单元测试、接口测试与前端 E2E 页面测试，每次提交触发自动构建与覆盖率报告生成。针对健康数据高并发写入场景开展压测，接口 P95 响应稳定在 400 毫秒内。问题追踪使用 Jira 看板，缺陷平均修复时长保持在八小时内，确保开发节奏和质量并重。

2.4.2 系统具体实现结果

系统上线后，用户能够在个人中心流畅录入血压、血糖、体重等多维指标，录入成功后页面实时刷新趋势图，折线与柱状切换动画流畅，即便在同时百人操作的高峰期也未出现卡顿。资讯首页按照后台设置的权重和推荐标记动态调整排序，热门内容点击率较预期提升了 27%，收藏与评论功能互动频次明显增加。后台健康模型配置界面以拖拽方式调整指标顺序，保存后前端录入表单即时同步，无需刷新页面即可生效。

消息中心推送链路稳定。用户在修改个人资料、收获系统公告或后台审核通过时可即时收到站内信提醒，已读未读切换在毫秒级完成。Kafka 队列监控数据显示，日均消息投递成功率保持 99.9%，未消费积压在夜间批处理窗口自动清空。限流与熔断策略多次在压测中验证拦截峰值流量，未对正常请求造成影响。

运维监控面板实时采集 CPU、内存、数据库连接和 Redis 命中率等指标，当资源使用超过阈值时自动扩容容器实例。两周运行数据显示，平均 CPU 占用 48%，内存占用 55%，系统保持稳定。生产环境日志经 ELK 汇总，关键错误告警通定期推送给值班人员，问题定位和恢复时间显著缩短。

三、 系统部署与测试工作情况

3.1 系统部署

系统部署采用 Docker 容器化方案，后端应用打包为基础镜像后，通过多阶段构建剥离编译产物，只保留可执行 Jar 与必要依赖，镜像体积压缩至百兆以内。前端项目在构建阶段完成静态资源打包，生成的 dist 目录被复制到 Nginx 镜像中，形成轻量级静态服务容器。数据库使用官方 MySQL 镜像，挂载主机数据卷以持久化存储，并在启动脚本中自动执行初始化 SQL，确保表结构一致。

容器编排采用 docker-compose，编排文件定义了 app、nginx、mysql、redis 四个服务，并通过自定义网络让容器彼此解析服务名直接通信，避免硬编码 IP。环境变量以 .env 文件集中管理，镜像启动后读取端口、数据库密码、JWT 密钥等配置，满足不同部署环境的可移植性。compose 提供的依赖顺序控制确保数据库与缓存服务先于后端启动，应用容器启动探针检测到依赖就绪后才进入正常运行状态。

上线流程中，开发者先在 CI 中完成镜像构建，并将版本号标签推送到私有仓库；服务器端拉取指定标签的镜像，运行 docker-compose up -d 完成滚动发布。旧容器被优雅停止，新容器启动后立即接管流量，实现无缝切换。日志通过 Docker volume 映射到主机目录，结合 logrotate 定期归档，为后期问题追溯与容量控制提供便利。

3.2 系统测试

系统进入测试阶段后首先运行单元测试套件，覆盖率统计显示后台核心服务层和控制层达到八十五行百分以上，数据访问层通过内存数据库替身验证了所有 CRUD 方法的边界条件。接口测试使用 Postman 脚本批量发起正反例请求，针对登录鉴权、健康数据录入、资讯分页查询等二十七个端点比对返回码与数据格式，未出现断言失败。前端方面引入 Cypress 进行端到端场景回放，自动登录后依次执行健康指标录入、资讯收藏与评论、消息阅读流程，浏览器端控制台无错误日志，视图渲染顺畅。

性能测试采用 JMeter 录制真实操作脚本，峰值并发五百用户持续十分钟压力下，API 网关 P95 响应维持在二百六十毫秒以内，CPU 占用未突破 50 个百分点。健康数据批量上传场景单次提交五百条记录，后端使用批处理与异步写入策略，数据库写吞吐达到每秒一万行，日志监控未捕获锁等待告警。前端资源加载通过 Lighthouse 审计，首屏渲染时间一秒八，交互就绪时间两秒二，图片懒加载与代码切割有效减少了首包体积。

安全测试阶段启用 OWASP ZAP 自动扫描，针对常见注入、跨站脚本、敏感信息泄漏未发现高危缺陷。人工渗透尝试利用 JWT 伪造与接口重放，网关限流和签名校验成功拦截非法请求。权限矩阵脚本遍历普通用户与管理角色的所有接口组合，未能越权访问敏感资源。数据一致性测试通过比较录入与查询结果的 CRC 校验值，确保健康记录计算逻辑在多线程环境下没有精度损失。

四、 项目管理相关工具使用情况

4.1 对项目过程的体会

版本控制平台最终选定 GitHub，整个开发周期都围绕着 Pull Request 流程展开。每一次功能提交前都需要先在 Issues 中创建对应任务，再由负责同学将分支推送并发起 Pull Request。模板要求填写动机、实现概要与验证步骤，关联的 Issue 自动在侧边栏展示，需求、代码与测试就此形成闭环。迭代冲刺期间，大家每天在 Projects 看板上拖动卡片，颜色从灰到绿的变化直观呈现进度，让剩余工时的压力始终可见。

需求评审和原型协作依赖 Figma，所有线框、配色与交互动效集中在单一文件。开发同学在 Figma 上直接评论交互疑问，设计同学即时响应，避免“最新版原型”在聊天群中反复传递。随着后台管理界面组件复用率提升，团队在 Figma 内建立了 Design System，将组件参数与前端常量一一映射，接口含义的理解差异明显减少。

持续集成采用 GitHub Actions 配合 Docker。代码推送即触发单元测试、静态扫描与镜像构建；构建结果通过 Action Badge 展示在仓库首页，状态一目了然。早期因缺乏专职运维，Runner 容器偶尔失效导致流水线静默中断，缺陷滞后被发现。后来在工作流中加入心跳监控与邮件告警，每次失败都会附带日志直达责任人，构建通过率遂与代码覆盖率并列为核心指标。

4.2 项目管理过程优点与不足

项目管理过程带来的最大收益首先体现在协作透明度上。GitHub Issues、Projects 与 Pull Request 互相关联，任何人都能在看板上一眼看到一项任务的创建日期、指派人、当前状态以及关联代码变更。信息壁垒被拆除后，需求讨论不再仅限于口头，而是在时间线上沉淀为可检索的记录，这让后来者追溯决策依据时省去大量沟通成本。同时，持续集成流水线把测试结果、构建日志与覆盖率报告自动贴到每一次合并申请下，代码质量从后台走向前台，开发者在提交瞬间就能感知缺陷并即时修复，团队内部形成了正向反馈循环。

然而高度流程化也带来摩擦。规范严格意味着进入门槛被抬高，新加入的成员要先适应看板规则、分支策略和模板格式，前几周的贡献速度明显放缓。CI 流水线的稳定性偶尔成为瓶颈，Runner 因资源耗尽而卡住整条发布链，当天其他功能的合并亦被连带阻滞。尽管后来增设了监控与告警，硬件与网络的偶发波动仍无法完全规避，团队在高并发提交的冲刺节点需要排队等待构建完成，这种节奏被动的停顿令部分成员感到焦虑。

协作工具对沟通方式的塑形也显而易见。Issue 评论、Review 对话与 Figma 批注让异步交流成为常态，减少了即时会议的次数，却在某些需要快速博弈的决策上显得节奏拖沓。当讨论陷入细节拉锯，文字往返数十条仍难以达成共识，大家又不得不回到在线会议以语音澄清。这样的往复切换在时间利用率上并不理想，也提示团队后续应对不同议题选择更合适的沟通介质。

五、 项目讨论与体会

需求评审的第一场讨论在综 c 的教室进行，围成一圈，显示屏上呈现着草稿和数据字典。大家都有共同的专业背景，却因为各自熟悉的技术细节不同，对接口粒度、字段命名乃至返回码都出现分歧。有同学坚持用布尔类型表达状态，另一些人主张统一以枚举提升可扩展性。争论的声音逐渐升高，直到负责原型设计的同学把 Figma 页面用共享桌面投到其他同学的屏幕，用交互示例演示字段含义在前端的直观效果，大家才意识到描述方式必须服务于界面呈现，这才找到共识。那天的会议持续了三个小时却无人倦怠，因为每个人都在用最熟悉的语言为同一个目标辩护。

进入编码阶段，团队采用一周一迭代的小步快跑节奏。版本控制仓库在最初几天频繁出现冲突，合并请求里夹杂格式化差异和临时调试语句，回滚一次就要耗费大半天。为此大家约定了代码规范，约定提交模板和分支命名，并在 nightly 构建脚本里加入格式化检查。第一次构建通过时终端输出一行绿色对勾，没有人鼓掌，但聊天群里冒出了整屏的 emoji。那一刻大家深刻体会到规则并非束缚，而是减少摩擦、让创造力聚焦到真正难题的工具。

健康数据可视化模块成为第二个高讨论点。实测数据接入后发现折线图在极端值附近抖动严重，视觉效果与设计稿落差明显。有人试图通过平滑算法掩盖跳变，但又担心失真；有人建议直接展示原始点阵，再在界面上提供数据说明。几经尝试，团队最终给出双通道方案：默认展示平滑曲线并在鼠标悬停时高亮原始点，既保证阅读流畅又不隐藏真实数据。方案确定后，负责前端渲染的同学凌晨两点在群里发来 demo 链接，曲线随光标闪动的细节让所有人屏息，那条线不只是数据，更像是一种对于精确与易读平衡的执念。

测试阶段的氛围出乎意料地紧张。由于没有运维同学专职监控环境，测试数据和正式数据混在同一台服务器，稍有不慎就会覆盖前一次成果。遇到关键缺陷时，大家只能在空教室手动拷贝数据库备份再重建环境。虽然繁琐，但这一过程让每位成员都亲手经历了从备份、还原到热修复的完整链路，对系统的依赖关系有了触摸式理解。一次深夜修复后，负责数据库的同学在日志里写下“每一次崩溃都是最好的教材”，第二天被贴在公告板上，成了团队的口头禅。

项目验收的前夜，所有模块已经跑通，只剩优化首页加载速度的任务。一位同学提出分离首屏脚本，另一位同学动手压缩图片并研究浏览器缓存策略。凌晨一点首页渲染时间终于从三秒降到一秒七，大家聚在实验楼灯火通明的走廊击掌庆祝。回想这些讨论与折腾，没有移动端、没有消息队列、没有专门运维，所有问题都得自己解决，正是这种“自己动手丰衣足食”的氛围让每个人都成长为比原先更完整的开发者。

六、 参考资料（见附件）

- 6.1 项目策划计划文档
- 6.2 系统需求规格说明书（SRS）
- 6.3 系统概要设计文档（SDD）
- 6.4 详细设计说明书
- 6.5 系统测试文档
- 6.6 用户文档说明书
- 6.7 项目管理过程文档（个人周报）